Giga-Embeddings: Mixture-of-Experts Encoders for High-Throughput Text Embeddings

Anonymous ACL submission

Abstract

We introduce Giga-Embeddings, a family of text embedding models designed to combine strong retrieval quality with efficient serving. Its largest member is a sparse 10B-parameter Mixture-of-Experts encoder that preserves substantial model capacity while activating only a fraction of it for each token. Across English, Russian, multilingual, and code MTEB benchmarks, the MoE model achieves the strongest aggregate performance within the family on all four evaluated suites. Under the evaluated vLLM serving configurations, its throughput is 28%, 25%, and 13% higher than that of the dense 3B model at input lengths of 512, 1024, and 2048 tokens, respectively. At 1024 tokens it reaches 114.5k tokens per second, 1.56–2.65× the throughput of the evaluated external systems. The compact 480M model reaches 425.7k tokens per second in the same batch-16 token-ID workload, while the dense 3B model has lower single-input latency than the MoE model. It also offers a Russian MTEB score comparable to FRIDA (70.96 vs. 70.95) with about 42% fewer reported parameters and supports multilingual embedding tasks. These measurements expose distinct quality, throughput, latency, and storage trade-offs rather than a uniformly superior serving architecture.

1 Introduction

Text embeddings support semantic search, retrieval-augmented generation, clustering, classification, and semantic textual similarity. MTEB (Muennighoff et al., 2023) and its multilingual expansion MMTEB (Enevoldsen et al., 2025) provide broad evaluation suites for these use cases. Recent embedding systems adapt decoder-only language models into bidirectional encoders and train them with large-scale contrastive objectives (BehnamGhader et al., 2024; Wang et al., 2024; Lee et al., 2025; Kolodin and Ianina, 2025). Although scaling the backbone generally improves representation quality, billion-parameter encoders are costly to store and serve.

Knowledge distillation offers one route to smaller encoders. For embedding models, supervision can be transferred by matching embeddings or by matching the teacher's relative similarity scores over candidate sets (Reimers and Gurevych, 2020; Xu et al., 2023; Zhao et al., 2026). The latter is attractive when the teacher and student have different hidden dimensions or architectures. It is also architecture-agnostic: a dense or sparse Mixture-of-Experts (MoE) teacher can supervise the same listwise objective through its final similarity scores.

We study a family of embedding systems that combines a high-capacity sparse encoder with smaller dense encoders, including a student trained with similarity-distribution distillation. The 10B MoE model uses an estimated 1.63B active weights per token under the counting convention in Appendix A and exceeds the dense 3B model's measured batch-16 throughput. The distilled 480M model targets compact deployment with Russian benchmark performance comparable to FRIDA and additional multilingual coverage. Our contributions are:

1. an openly released model family spanning dense 480M and 3B encoders and a sparse 10B encoder;

2. a four-suite MTEB evaluation in which the 10B model has the highest reported aggregate scores within the family and the highest Russian score in our contextual external comparison;

3. a compact encoder with a reported Russian MTEB score comparable to FRIDA (70.96 vs. 70.95) at about 42% fewer parameters (480M vs. 823M), together with English, multilingual, and code coverage;

4. a serving characterization separating batch throughput from single-input latency: the 10B model has 13–28% higher batch-16 throughput than 3B, whereas 3B has lower batch-1 latency and 480M has the highest measured throughput. These model-level comparisons do not isolate the causal effect of sparsity.

Direct checkpoint links are withheld during anonymous review and will be restored in the camera-ready version.

2 Related Work

General-purpose text embeddings. Sentence-BERT (Reimers and Gurevych, 2019) and SimCSE (Gao et al., 2021) established bi-encoder and contrastive recipes, subsequently scaled by E5, GTE, and BGE (Wang et al., 2022; Li et al., 2023; Xiao et al., 2023). Instruction-conditioned systems such as INSTRUCTOR, E5-Mistral, and NV-Embed (Su et al., 2023; Wang et al., 2024; Lee et al., 2025) adapt language models to embedding tasks. Qwen3-Embedding (Zhang et al., 2025) uses the same architecture family as our dense encoders.

Embedding distillation. Classical distillation matches output distributions (Hinton et al., 2015). Representation transfer and contrastive self-distillation have been studied for sentence embeddings (Reimers and Gurevych, 2020; Xu et al., 2023), while retrieval methods transfer margins or listwise similarities (Hofstätter et al., 2020; Ren et al., 2021; Lee et al., 2024). KaLM-Embedding-V2 and jina-embeddings-v5-text combine contrastive supervision with distillation (Zhao et al., 2026; Akram et al., 2026). Our recipe likewise uses an established KL objective on similarity distributions; the objective itself is not a novelty claim.

MoE embedding models. MoE models increase total capacity while activating a subset of experts per token (Shazeer et al., 2017; Fedus et al., 2022; Jiang et al., 2024). Nomic Embed v2 (Nussbaum and Duderstadt, 2025) is a previously released general-purpose MoE text embedding model. MoEE (Li and Zhou, 2025) constructs embeddings from routing weights and hidden states of existing MoE language models without additional fine-tuning. Our study evaluates a trained bidirectional MoE encoder alongside dense models in the same released family.

Table 1: Core training settings; archived branch variations are detailed in Appendix B. Fine-tuning and multitask batches contain 8,192 candidate texts.

Setting	Pretrain	Finetune	Multitask
Global batch size	16,384	1,024	1,024
Candidates/sample	in-batch	1 pos. + 7 neg.	1 pos. + 7 neg.
In-batch negatives	yes	no	no

3 Model and Training Recipe

3.1 Encoder adaptation

Following the earlier Giga-Embeddings recipe (Kolodin and Ianina, 2025), the model family adapts decoder-only language models for embedding generation. All three encoders are initialized from proprietary GigaChat pre-training checkpoints. The architecture-family names below describe their network designs rather than public initialization checkpoints. The causal attention mask is replaced by a fully visible bidirectional mask, as in LLM2Vec (BehnamGhader et al., 2024), so that each token can use both left and right context. We use mean pooling to aggregate token states into a fixed-dimensional vector and normalize the resulting vector before cosine-similarity scoring. The instruction templates follow the earlier Giga-Embeddings recipe (Kolodin and Ianina, 2025) and distinguish asymmetric retrieval inputs from symmetric semantic-similarity inputs. The 480M, 3B, and 10B-A1.8B checkpoints and instruction set are publicly available; direct repository links are withheld during anonymous review. The released model cards specify the MIT licence for all three checkpoints.

The 480M, 3B, and 10B-A1.8B encoders produce embeddings with dimensions 1024, 2048, and 1536, respectively. The 480M and 3B models use dense bidirectional encoders in the Qwen3 architecture family (Zhang et al., 2025), while the largest model is a bidirectional DeepSeekMoE-style encoder (Dai et al., 2024) with 10B total parameters, 64 routed experts, one shared expert, and top-4 routing. The dense encoders have 28 and 36 layers with hidden sizes 1024 and 2048; the MoE encoder has 26 layers with hidden size 1536, sigmoid router scores, and 1280-dimensional expert feed-forward layers. We retain the released model identifier "10B-A1.8B"; our explicit active-weight estimate is approximately 1.63B, as detailed with the checkpoint configurations in Appendix A. We use "480M", "3B", and "10B-A1.8B" as short model-

family names.

3.2 Three-stage contrastive training

As in the first Giga-Embeddings release (Kolodin and Ianina, 2025), training has three stages: broad contrastive pre-training, retrieval fine-tuning with hard negatives, and multitask fine-tuning for retrieval, classification, clustering, and semantic similarity. Embedding training uses open-source data from the Nemotron (Babakhin et al., 2025) and KaLM (Zhao et al., 2026) recipes. Appendix B identifies the corresponding public source collections and their published roles. The proprietary component is the GigaChat backbone pre-training, whereas the embedding-stage data are open-source. Across all stages, the learning rate starts at 3×10^{-5} and follows a cosine decay schedule with a 1% warmup fraction and a minimum learning rate of 10^{-7}. We use weight decay of 0.01, Adam coefficients $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and an InfoNCE temperature of 0.02. Fine-tuning runs for one epoch and multitask training for three epochs. The stage-specific settings are summarized in Table 1. The recovered compact F2LLM pre-training branch uses global batch 1024 rather than the core pre-training value 16,384; its membership in the final merge is not established.

Our contrastive-loss formulation follows Qwen3-Embedding (Zhang et al., 2025). Pre-training uses in-batch InfoNCE, includes query–query and document–document contrastive terms, and masks detected false negatives. Fine-tuning and multitask training disable in-batch negatives and use the explicit eight-candidate groups described in Table 1. For a batch of B queries, let $\mathcal{C}_i$ be the fixed candidate set for query q_i, let $p(i)$ index its positive candidate, and let s_{ij} be the student cosine similarity.

We use batch-mean InfoNCE (van den Oord et al., 2018):

$$\mathcal{L}_{\mathrm{NCE}} = -\frac{1}{B}\sum_{i=1}^{B}\log\frac{\exp(s_{i,p(i)}/\tau)}{\sum_{j\in\mathcal{C}_i}\exp(s_{ij}/\tau)}. \quad (1)$$

Hard negatives for retrieval fine-tuning are mined from a predefined retrieval-rank range.

3.3 Similarity-distribution distillation

We apply similarity-distribution distillation exclusively to the 480M model during retrieval fine-tuning and multitask fine-tuning. The teacher is a 10B MoE encoder initialized from a proprietary GigaChat pre-training checkpoint. Teacher query and candidate vectors are precomputed and cached; training forms similarity targets from these vectors rather than executing the teacher online. Its pre-training stage uses only the contrastive objective, and the 3B and 10B-A1.8B models are trained without distillation. For both distilled stages, $\lambda = 0.3$, $\tau_{\mathrm{KD}} = 0.05$, and the contrastive temperature is 0.02. Because the objective aligns distributions over query–candidate similarity scores rather than embedding coordinates, the teacher and student may use different embedding dimensions. By contrast, direct MSE or cosine-similarity alignment between teacher and student embedding vectors requires equal dimensions or an additional learned projection. Let t_{ij} be the teacher cosine similarity for the same support $\mathcal{C}_i$. Teacher and student distributions are

$$P_T(j \mid q_i) = \frac{\exp(t_{ij}/\tau_{\mathrm{KD}})}{\sum_{k\in\mathcal{C}_i}\exp(t_{ik}/\tau_{\mathrm{KD}})}, \quad (2)$$

$$P_S(j \mid q_i) = \frac{\exp(s_{ij}/\tau_{\mathrm{KD}})}{\sum_{k\in\mathcal{C}_i}\exp(s_{ik}/\tau_{\mathrm{KD}})}. \quad (3)$$

The distillation term is a batch mean over identical supports:

$$\mathcal{L}_{\mathrm{KD}} = \frac{1}{B}\sum_{i=1}^{B} D_{\mathrm{KL}}(P_T(\cdot \mid q_i) \,\|\, P_S(\cdot \mid q_i)). \quad (4)$$

The combined objective is

$$\mathcal{L} = \lambda\mathcal{L}_{\mathrm{KD}} + (1-\lambda)\mathcal{L}_{\mathrm{NCE}}. \quad (5)$$

The KL term transfers the teacher's relative similarity structure over each candidate set, while InfoNCE preserves direct contrastive supervision. Because the loss depends only on final similarity scores, it does not require access to the teacher's internal architecture or routing decisions.

4 Evaluation

4.1 Evaluation protocol

For the released checkpoints, we report the official MTEB Leaderboard's benchmark-level `meanTask` aggregates retrieved on 2026-10-29. The four suites contain 41 English, 23 Russian, 131 multilingual, and 12 code tasks; each task receives equal weight in the suite-level mean. The Russian suite follows ruMTEB (Snegirev et al., 2025), while the multilingual suite follows MMTEB (Enevoldsen et al., 2025). The public

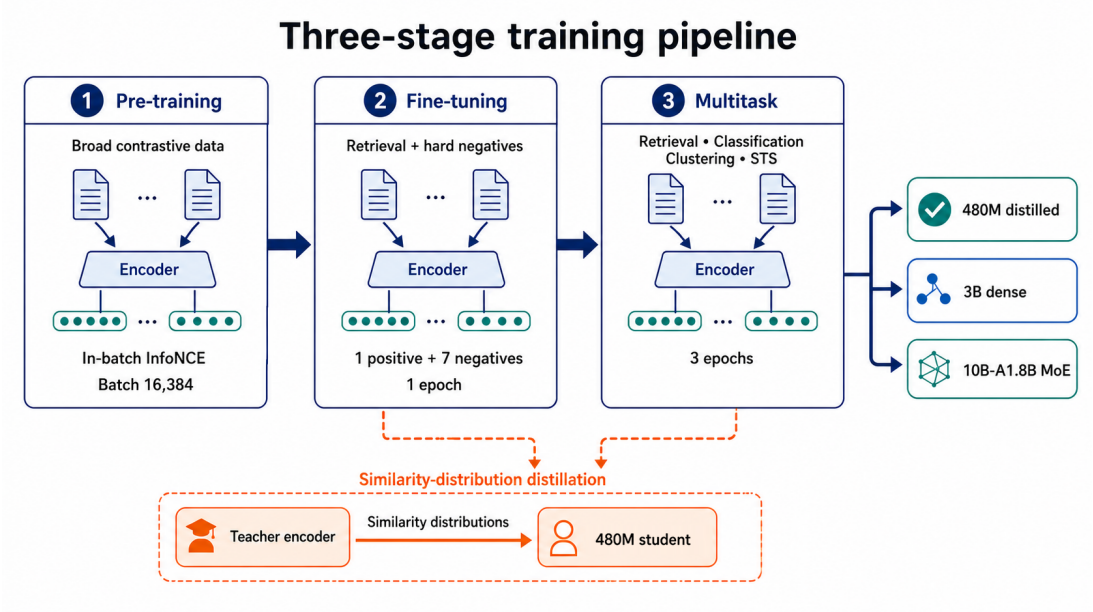


Figure 1: Core three-stage embedding training pipeline. All models undergo contrastive pre-training, retrieval fine-tuning, and multitask fine-tuning. During the latter two stages, only the 480M student additionally matches similarity distributions from a 10B MoE teacher.

Table 2: Model architectures; active counts follow Appendix A.

Type	Total	Active	Dim.
Dense	480M	480M	1024
Dense	3B	3B	2048
MoE	10B	∼1.63B	1536

Table 3: MTEB scores (%). Leaderboard `meanTask`, 2026-10-29. †: reaggregated archive (Appendix F). Bold: family maximum.

Scale	Dim.	English	Russian	Multilingual	Code
480M	1024	69.52	70.96	56.80†	72.87
3B	2048	71.90	74.55	63.89	76.94
10B-A1.8B	1536	**72.23**	**74.99**	**65.64**	**78.43**

Table 4: Offline token-ID throughput (tokens/s): one H100 80 GB per run, vLLM 0.24.0, BF16, batch 16. Seven timed repeats after three warm-ups; relative: MoE = 1 at 1024 tokens.

Model (vLLM)	512 tokens	1024 tokens	2048 tokens	Relative
Qwen3-Embedding-4B	70.1k	73.2k	71.2k	0.64×
F2LLM-v2-8B	43.2k	43.4k	42.6k	0.38×
nvidia/llama-embed-nemotron-8b	42.6k	43.2k	41.7k	0.38×
Ours: 480M	**356.2k**	**425.7k**	**437.1k**	**3.72×**
Ours: 3B	87.9k	91.5k	90.4k	0.80×
Ours: 10B-A1.8B	112.6k	114.5k	102.3k	1.00×

Table 5: Contextual comparison with compact models on MTEB benchmark suites (%). Public baselines use official MTEB Leaderboard `meanTask` values retrieved on 2026-08-08; our result is from Table 3. Scores use different evaluation settings; a dash denotes an unavailable score.

Model	Params	Dim.	Russian	English	Multilingual	Code
sergeyzh/BERTA	128M	768	69.39	—	—	—
google/embeddinggemma-300m	308M	768	65.19	69.67	61.15	68.76
codefuse-ai/F2LLM-v2-0.6B	596M	1024	65.97	69.97	62.74	77.41
Qwen/Qwen3-Embedding-0.6B	596M	1024	64.29	70.47	64.34	75.42
ai-forever/FRIDA	823M	1536	70.95	—	—	—
Ours: 480M	480M	1024	70.96	69.52	56.80†	72.87

snapshot has no complete multilingual aggregate for 480M. Its † value is a CPU reaggregation of an archived, mixed-version score cache attributed to that model, not a new inference run or an indepen-

Table 6: Contextual comparison with larger models on MTEB benchmark suites (%). Public baselines use official MTEB Leaderboard `meanTask` values retrieved on 2026-08-08; our results are from Table 3. Scores use different evaluation settings.

Model	Params	Dim.	Russian	English	Multilingual	Code
codefuse-ai/F2LLM-v2-1.7B	1.72B	2048	68.52	71.63	65.21	78.76
codefuse-ai/F2LLM-v2-4B	4.02B	2560	69.46	72.41	67.06	80.15
Qwen/Qwen3-Embedding-4B	4.02B	2560	70.13	74.61	69.45	80.07
Qwen/Qwen3-Embedding-8B	7.57B	4096	71.42	75.23	70.58	80.69
codefuse-ai/F2LLM-v2-8B	7.57B	4096	70.57	72.86	68.09	80.16
codefuse-ai/F2LLM-v2-14B	13.99B	5120	70.90	73.08	68.74	80.75
Ours: 3B	3B	2048	74.55	71.90	63.89	76.94
Ours: 10B-A1.8B	10B	1536	74.99	72.23	65.64	78.43

dently checkpoint-linked public score. This gives 56.80; the earlier 56.97 is withdrawn because the recovered notebook has a stale-score aggregation defect (Appendix F). The distillation comparison retains the original run-level ablation scores and is reported separately from these public-release aggregates. For 480M Russian, the original 70.98 and public 70.96 are reproducible from identical task results: test-preferred averaging gives 70.978526, whereas including validation and test for the two MASSIVE tasks gives 70.959130 (Appendix E).

At two bytes per weight (BF16 or FP16), the 480M, 3B, and 10B-A1.8B models require approximately 0.97, 6.30, and 20.95 GB of weight storage, respectively (decimal GB). The distilled model therefore uses about 4.6% as much weight storage as the MoE model. These theoretical estimates exclude activations and other runtime state.

4.2 Aggregate results

Table 3 presents task-macro scores for all three models. The 480M distilled model reaches 69.52 on English, 70.96 on Russian, and 72.87 on Code in the public snapshot; its reaggregated archived Multilingual value is 56.80. Using the underlying, unrounded public aggregates, its English, Russian, and Code gaps to 3B are 2.39, 3.59, and 4.07 points. The Multilingual gap is approximately 7.09 points using the archived value, subject to the checkpoint-linkage limitation above.

The 10B-A1.8B model achieves the highest score within the family on every benchmark. Using unrounded public aggregates, the differences from 3B are +0.33 on English, +0.43 on Russian, +1.75 on Multilingual, and +1.49 on Code. Because each model is evaluated once, the sub-one-point differences on English and Russian should be interpreted cautiously.

Table 7: Original single-run training variants (%). Differences in initialization and microbatch size prevent interpreting this as a matched KD-only control.

Model	English	Russian	Code
480M, no distillation	69.43	70.86	72.65
480M, distilled	69.52	70.98	72.87

4.3 Descriptive distillation comparison

Table 7 reports two single-run 480M training variants, using the original test-preferred Russian aggregation. The observed English, Russian, and Code differences are +0.09, +0.12, and +0.22 points, respectively. Recovered records name different merged starting checkpoints and microbatch sizes (128 with KD, 64 without), although both use global batch 1024. The current initialization files differ in 308 of 310 tensors; their historical training-time hashes were not retained. Training-row identity, historical inference settings, and merge equivalence are not established, so these differences do not isolate a causal KD effect. No multilingual no-KD result or repeated-seed series was recovered.

4.4 Inference throughput

Recovered timings use one H100 80 GB HBM3 per run, BF16, and vLLM 0.24.0. Inputs are synthetic random token IDs of exact length, without padding; each cell has three warm-ups and seven synchronous timed repeats. Throughput is BL divided by median encode-call wall time, excluding model loading, tokenization, and input generation. The script, software versions, configuration differences, and checkpoint-linkage scope are detailed in Appendix C.

At batch 16, the MoE configuration exceeds 3B throughput by 28%, 25%, and 13% across the three lengths, while 480M has the highest throughput overall. At batch 1, however, 3B has lower latency

Table 8: Median offline encode-call latency at 1024 tokens (ms; seven timed repeats). Batch 16 is whole-batch completion time, not per-query latency.

Model	Batch 1	Batch 16
480M	7.29	38.49
3B	13.24	179.05
10B-A1.8B	30.16	143.13

than MoE (Table 8); the preferred checkpoint therefore depends on batching and quality requirements. These are offline encoder-call timings, without network or queueing latency. The models and engine profiles differ, including an eager, patched MoE profile, so the measurements do not isolate sparsity or establish an equal-quality advantage. Valid vLLM worker/device memory measurements are incomplete; Appendix D separates available allocator measurements from weight-storage estimates.

4.5 External baseline comparison

Tables 5 and 6 compare Giga-Embeddings with public results from the MTEB Leaderboard. Differences in prompts and evaluation settings make these contextual rather than controlled same-run comparisons. Our 10B-A1.8B model has the highest reported Russian score in this comparison, while Qwen3-Embedding-8B has the highest English and Multilingual scores and F2LLM-v2-14B the highest Code score. Qwen3-Embedding-4B also scores higher than our MoE model on English, Multilingual, and Code; the throughput comparison therefore does not establish an advantage at equal embedding quality. The compact model illustrates parameter efficiency at comparable reported Russian quality: it scores 70.96 against FRIDA's 70.95 with about 42% fewer parameters, using the reported model sizes of 480M and 823M. The same checkpoint also supports English, multilingual, and code embedding tasks.

5 Conclusion

We present Giga-Embeddings, a family of dense and sparse bidirectional text encoders. The 10B-A1.8B model has the highest reported aggregate scores within the family on four MTEB suites and 13–28% higher measured batch-16 throughput than the dense 3B model. The 480M model is fastest in the token-ID benchmark, while 3B has lower batch-1 latency than MoE; the MoE's larger full checkpoint also requires more weight storage. The external comparisons show a quality trade-off: the MoE model has a higher reported Russian score, whereas Qwen3-Embedding-4B has higher English, Multilingual, and Code scores. The 480M model combines a reported Russian MTEB score comparable to FRIDA with about 42% fewer parameters and additional multilingual coverage. Together, the checkpoints illustrate distinct trade-offs between benchmark performance, measured token throughput, and model storage for embedding workloads.

6 Limitations

Our benchmark results report a single run per model, so small score differences do not have uncertainty estimates. Serving results use synthetic token IDs on one GPU SKU across separate sessions, with only seven timed repeats per setting; they are not natural-text or online tail-latency measurements. Engine profiles differ, and valid device-wide peak memory is unavailable. The backbone pre-training checkpoints are proprietary, which limits its reproduction from raw data. Preserved records do not establish complete initialization, teacher-cache, and merge lineage, or the historical MoE balancing objective. We report no overlap between embedding-training and evaluation examples; a detailed record of the checking procedure was not located in the inspected artifacts. The KD comparison has training and packaging confounds and does not establish a causal distillation gain. The 480M Multilingual value is reaggregated from an archived mixed-version cache without an immutable checkpoint binding. Finally, aggregate MTEB scores can hide variation across individual tasks and languages. Future evaluations should include matched controls, frozen evaluation provenance, natural-text workloads, and robust latency and memory measurements.

References

Mohammad Kalim Akram, Saba Sturua, Nastia Havriushenko, Quentin Herreros, Michael Günther, Maximilian Werk, and Han Xiao. 2026. jina-embeddings-v5-text: Task-targeted embedding distillation. *arXiv preprint arXiv:2602.15547*.

Yauhen Babakhin, Radek Osmulski, Ronay Ak, Gabriel Moreira, Mengyao Xu, Benedikt Schifferer, Bo Liu, and Even Oldridge. 2025. Llama-Embed-Nemotron-8B: A universal text embedding model for multilingual and cross-lingual tasks. *arXiv preprint arXiv:2511.07025*.

Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and

Siva Reddy. 2024. LLM2Vec: Large language models are secretly powerful text encoders. In *Proceedings of the First Conference on Language Modeling (COLM)*.

Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, et al. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1280–1297.

Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik Krzemiński, Genta Indra Winata, et al. 2025. MMTEB: Massive multilingual text embedding benchmark. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6894–6910.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Sebastian Hofstätter, Sophia Althammer, Michael Schröder, Mete Sertkan, and Allan Hanbury. 2020. Improving efficient neural ranking models with cross-architecture knowledge distillation. *arXiv preprint arXiv:2010.02666*.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, and 7 others. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088*.

Egor Kolodin and Anastasia Ianina. 2025. GigaEmbeddings — efficient Russian language embedding model. In *Proceedings of the 10th Workshop on Slavic Natural Language Processing (Slavic NLP 2025)*, pages 17–24, Vienna, Austria. Association for Computational Linguistics.

Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. 2025. NV-Embed: Improved techniques for training LLMs as generalist embedding models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Jinhyuk Lee, Zhuyun Dai, Xiaoqi Ren, Blair Chen, Daniel Cer, Jeremy R. Cole, Kai Hui, Michael Boratko, Rajvi Kapadia, Wen Ding, et al. 2024. Gecko: Versatile text embeddings distilled from large language models. *arXiv preprint arXiv:2403.20327*.

Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. 2023. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*.

Ziyue Li and Tianyi Zhou. 2025. Your mixture-of-experts LLM is secretly an embedding model for free. In *The Thirteenth International Conference on Learning Representations*.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. 2023. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, pages 2014–2037.

Zach Nussbaum and Brandon Duderstadt. 2025. Training sparse mixture of experts text embedding models. *arXiv preprint arXiv:2502.07972*.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992.

Nils Reimers and Iryna Gurevych. 2020. Making monolingual sentence embeddings multilingual using knowledge distillation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4512–4525.

Ruiyang Ren, Yingqi Qu, Jing Liu, Wayne Xin Zhao, Qiaoqiao She, Hua Wu, Haifeng Wang, and Ji-Rong Wen. 2021. RocketQAv2: A joint training method for dense passage retrieval and passage re-ranking. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2825–2835.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Artem Snegirev, Maria Tikhonova, Anna Maksimova, Alena Fenogenova, and Alexander Abramov. 2025. The Russian-focused embedders' exploration: ruMTEB benchmark and Russian embedding model design. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 236–254.

Hongjin Su, Weijia Shi, Jungo Kasai, Yizhong Wang, Yushi Hu, Mari Ostendorf, Wen-tau Yih, Noah A. Smith, Luke Zettlemoyer, and Tao Yu. 2023. One

embedder, any task: Instruction-finetuned text embeddings. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 1102–1121.

Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*.

Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.

Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. 2024. Improving text embeddings with large language models. *arXiv preprint arXiv:2401.00368*.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2023. C-Pack: Packed resources for general Chinese embeddings. *arXiv preprint arXiv:2309.07597*.

Jiahao Xu, Wei Shao, Lihui Chen, and Lemao Liu. 2023. DistillCSE: Distilled contrastive learning for sentence embeddings. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8153–8165.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Xinping Zhao, Xinshuo Hu, Zifei Shan, Shouzheng Huang, Yao Zhou, Xin Zhang, Zetian Sun, Zhenyu Liu, Dongfang Li, Xinyuan Wei, Youcheng Pan, Yang Xiang, Meishan Zhang, Haofen Wang, Jun Yu, Baotian Hu, and Min Zhang. 2026. KaLM-Embedding-V2: Superior training techniques and data inspire a versatile embedding model. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

A Released Checkpoint Configurations

The public release configurations specify 28 layers, hidden size 1024, feed-forward size 3072, 16 attention heads, 8 key/value heads, and head dimension 64 for the 480M encoder. The 3B encoder has 36 layers, hidden size 2048, feed-forward size 11008, 16 attention heads, 8 key/value heads, and head dimension 128. Both use the Qwen3 bidirectional architecture family and a vocabulary of 128,256 tokens.

The MoE encoder uses the DeepSeek-V3 bidirectional architecture family with 26 layers, hidden size 1536, 32 attention heads, and the same vocabulary size. Its first layer has a dense feed-forward block of intermediate size 8960; the remaining 25 layers use 64 routed experts with intermediate size 1280, top-4 selection, and one shared expert. The released router configuration specifies sigmoid scores, `noaux_tc` selection, one expert group, normalization of selected routing weights, and a routing scale of 1.0. Multi-head latent attention uses a key/value low-rank dimension of 512, non-rotary and rotary query/key head dimensions of 128 and 64, and a value-head dimension of 192. These are release-configuration settings; the router implementation name alone does not establish the training-time balancing objective.

The released safetensors metadata give total parameter counts of 483,717,632, 3,150,605,312, and 10,475,534,400 for the 480M, 3B, and MoE models, respectively. The MoE routed expert matrices contain

$$\begin{aligned} N_{\text{routed}} &= 25 \cdot 64 \cdot 3 \cdot 1536 \cdot 1280 \\ &= 9{,}437{,}184{,}000. \end{aligned} \tag{6}$$

weights, accounting for the gate, up, and down projections of each expert. Our active-weight estimate counts all non-routed weights, including shared experts and the embedding table, plus four of the 64 routed experts per token:

$$N_{\text{active}} = N_{\text{total}} - N_{\text{routed}} + \frac{4}{64} N_{\text{routed}} \tag{7}$$

$$= 1{,}628{,}174{,}400. \tag{8}$$

This convention estimates active weights rather than FLOPs, GPU residency, or peak memory.

B Public Training Data Sources

The source recipes are Nemotron (Babakhin et al., 2025) and KaLM (Zhao et al., 2026). KaLM publishes a pre-training collection (HIT-TMG) and a fine-tuning collection (KaLM-Embedding). The Nemotron report describes a retrieval-pretraining mixture derived from Nemotron-CC-v2 and a separate fine-tuning mixture; its public fine-tuning release is Embed Nemotron Dataset V1. These references identify the public source collections and distinguish their published roles; their published sizes are not estimates of our post-filtering stage-specific sample counts.

Recovered experiment manifests list 25 KaLM pre-training files, 62 KaLM fine-tuning files, 14 Nemotron fine-tuning files, and 123 inputs in

Table 9: Archived compact-model training counts. Prepared rows and consumed samples have different meanings; rows must not be summed as a unique final training mixture.

Branch/stage	Prepared rows	Consumed
Giga pretrain	91,030,382	91,029,504
KaLM pretrain	299,034,744	299,024,384
F2LLM pretrain	10,597,526	10,597,376
Giga finetune	1,131,959	1,131,520
Nemotron finetune	3,355,596	3,354,624
KaLM finetune	2,797,672	2,797,568
F2LLM finetune	3,511,784	3,511,296
Multitask	2,439,873	7,317,504

one multitask manifest. The 14 Nemotron entries are EmotionClassification, FEVER, GooAQ, HotpotQA, MAmmoTH2, MIRACL, MSMARCO, NaturalQuestions, NFCorpus, PAQ, SciFact, SQuAD, SyntheticClassificationData, and TriviaQA. Additional manifests describe earlier-recipe and F2LLM branches, but the full merge lineage establishing which branches enter each final release was not recovered. These counts describe file inventories, not example counts or independently verified release mixtures. Table 9 adds prepared-dataset sizes reported in preserved compact-model logs and consumed-sample counters recovered from their checkpoints. Prepared rows are the logged dataset size after preparation; consumed samples include repeated epochs and are not unique-example counts. These measurements describe particular development branches rather than a reconstructed final-release mixture. The multitask row belongs to a completed August 3 run, not the missing August 4 KD or August 7 no-KD checkpoints. KaLM fine-tuning counts are confirmed for no-KD and a separate KaLM-12B-teacher run, not the missing 10B-teacher log. Giga and F2LLM have completed compact training branches, but executed merges linking all branch outputs to each release remain unverified.

The 14 preserved compact completed-run argument records use seed/data seed 42, dataset shuffling, no interleave-probability vector, no packing, and deletion of overlength examples; maximum lengths are 512 for pre-training/fine-tuning and 2048 for multitask training. The F2LLM pretraining branch uses global batch 1024, whereas the Giga and KaLM pre-training branches use 16,384. Requested per-source sample selectors do not establish unique counts or effective post-filtering source weights. RU/EN/other proportions, immutable input revisions, and final-release sampling weights remain unavailable. We report no training/evaluation example overlap; the inspected archive does not contain the detailed checking procedure.

C Recovered Serving Protocol

The raw timing artifacts record H100 80 GB HBM3 hardware, BF16, Python 3.12.13, PyTorch 2.11.0+cu129, CUDA runtime 12.9, Transformers 5.13.0, and vLLM 0.24.0. Each run uses one GPU; the runs occurred across separate sessions on the same GPU SKU rather than a verified single physical device. The NVIDIA driver version and complete engine startup output were not retained.

The benchmark generates token IDs on CPU with a fixed generator seed of 20260731. For each length L and batch size B, IDs are sampled uniformly from $[3, V)$; the first and last positions are set to IDs 1 and 2, without padding. The vocabulary bound is 128,000 for our models and Nemotron and 151,643 for Qwen3-Embedding-4B and F2LLM-v2-8B. The inputs are neither natural texts nor common semantic inputs across different tokenizers. For timed repeat r, the seed is $20260731 + 100000r + L + B$; warm-up w uses $20260731 + w + L + B$.

Each (B, L) setting has three warm-ups followed by seven timed calls. Input generation precedes timing; a client-side CUDA synchronization precedes the wall-clock timer, which covers the synchronous embedding call and a subsequent synchronization. The embedding call includes engine scheduling and input/output handling, while loading, tokenization, and input generation are excluded. Throughput is $BL/\operatorname{median}(t_1, \ldots, t_7)$. Pooling is mean for our models and Nemotron and last-token for Qwen3 and F2LLM, followed by L2 normalization.

The recovered configuration sets the pooling runner, embedding conversion, maximum length 2048, maximum sequence count 16, GPU memory utilization 0.90, and prefix caching disabled. Our dense encoders use bidirectional Qwen3 overrides. The MoE profile uses DeepSeek-V3 overrides, an encoder-only attention patch, eager execution, disabled FlashInfer autotuning, disabled MLA backend, and in-process execution; a DeepSeek rotary custom operation is disabled in favor of its native path. Thus, the profiles differ in architecture and implementation. Unrecorded engine defaults are not treated as recovered effective settings, and no

profiling evidence isolates the cause of the 2048-token throughput decline.

The compact timing file uses a legacy "600M" label and points to the checkpoint now containing 483,717,632 parameters. The server audit records a current weight-file SHA-256 matching the public 480M release, supporting this mapping. Contemporaneous weight hashes are absent from the historical timing logs; the original 3B/10B timing checkpoint paths are no longer present.

D Memory Measurement Scope

Table 11 reports recovered peak PyTorch allocated-memory counters at batch 16, in GiB (2^{30} bytes). The counters include live weights and buffers but do not measure all device-used memory or the reserved-memory peak. Transformers and vLLM are separate backend measurements, so Transformers memory must not be interpreted as vLLM memory at the reported vLLM throughput. Dense vLLM logs contain zero-valued client-process counters that do not capture the worker allocations; these are marked unavailable rather than zero memory. The in-process MoE profile has nonzero allocated-memory counters, but device-wide peak memory remains unavailable for all three vLLM configurations.

E Russian Score Reconciliation

The archived local and public 480M Russian results have identical dataset revisions and subset/split scores on all 23 tasks. For MassiveIntentClassification, the test and validation scores are 0.826429 and 0.823463; for MassiveScenarioClassification they are 0.898184 and 0.892228. The original notebook prefers test when present, producing a 23-task mean of 70.978526%. Averaging the validation and test scores of those two tasks gives 70.959130%, matching the public aggregate, a difference of 0.019396 percentage points. No changed task-level score is required to explain the displayed 70.98 versus 70.96. The original KD/no-KD comparison uses the test-preferred protocol for both variants; its small score differences must not be recomputed by mixing one variant's original score with the other's public aggregate.

F Archived Multilingual Score Reaggregation

The archived 480M Multilingual cache contains all 131 tasks and 46,933 subset/split score cells selected by the preserved benchmark manifest. Recomputing each task mean over its selected cells and then averaging tasks gives 56.799822%, reported as $56.80^{\dagger}$. Dataset revision strings match the selected-cell manifest; the cache combines MTEB versions 2.12.30, 2.15.4, and 2.18.0 and lacks an immutable model revision. Thus, this is a reanalysis of stored scores attributed to 480M, not a new evaluation or a verified score for a frozen release checkpoint.

The recovered notebook fails to reset its score variable when a task has no recognized split name. For LEMBPasskeyRetrieval, whose splits are named `test_256` through `test_32768`, it can reuse 0.93308 from the preceding LegalBenchCorporateLobbying task rather than the actual eight-split mean 0.6625. With the archived file order, replay gives 56.970884%, reproducing the previously reported 56.97 numerically. The stale LEMB value contributes +0.206550 percentage points, while other split-selection differences contribute -0.035487, for a net +0.171062. Paper-time invocation and input hashes were not retained, so this numerical match does not establish the historical cause of the original figure; we withdraw that figure rather than retain an aggregation susceptible to the defect.

The corresponding selected-cell means for 3B and MoE are 63.890167% and 65.637226%, matching their public aggregates. Their 46,933 selected cells each match the preserved public results. Although the same notebook defect exists, it does not invalidate those independently sourced public means.

Table 10: Recovered offline latency (ms). Each setting has seven timed samples; p95 is recomputed by linear interpolation at rank $(n-1)0.95$ from those same samples and is descriptive, not a reliable tail-latency or online-SLO estimate. B16 denotes completion of the whole batch.

Model	Tokens	B1 p50	B1 p95	B16 p50	B16 p95
480M	512	7.14	8.70	23.00	23.25
	1024	7.29	7.33	38.49	38.70
	2048	7.39	7.47	74.96	76.05
3B	512	13.27	15.06	93.19	96.29
	1024	13.24	13.26	179.05	186.30
	2048	24.16	24.86	362.43	365.44
10B-A1.8B	512	30.18	30.31	72.76	72.86
	1024	30.16	30.28	143.13	152.80
	2048	30.64	30.88	320.29	324.68

Table 11: Backend-specific peak allocated memory at B16 (GiB). A dash denotes an invalid or unavailable worker counter, not zero memory.

Model/tokens	Transformers	vLLM
480M / 512	1.136	—
480M / 1024	1.339	—
480M / 2048	1.746	—
3B / 512	6.635	—
3B / 1024	7.265	—
3B / 2048	8.523	—
10B / 512	20.294	20.637
10B / 1024	21.030	21.561
10B / 2048	22.503	21.561